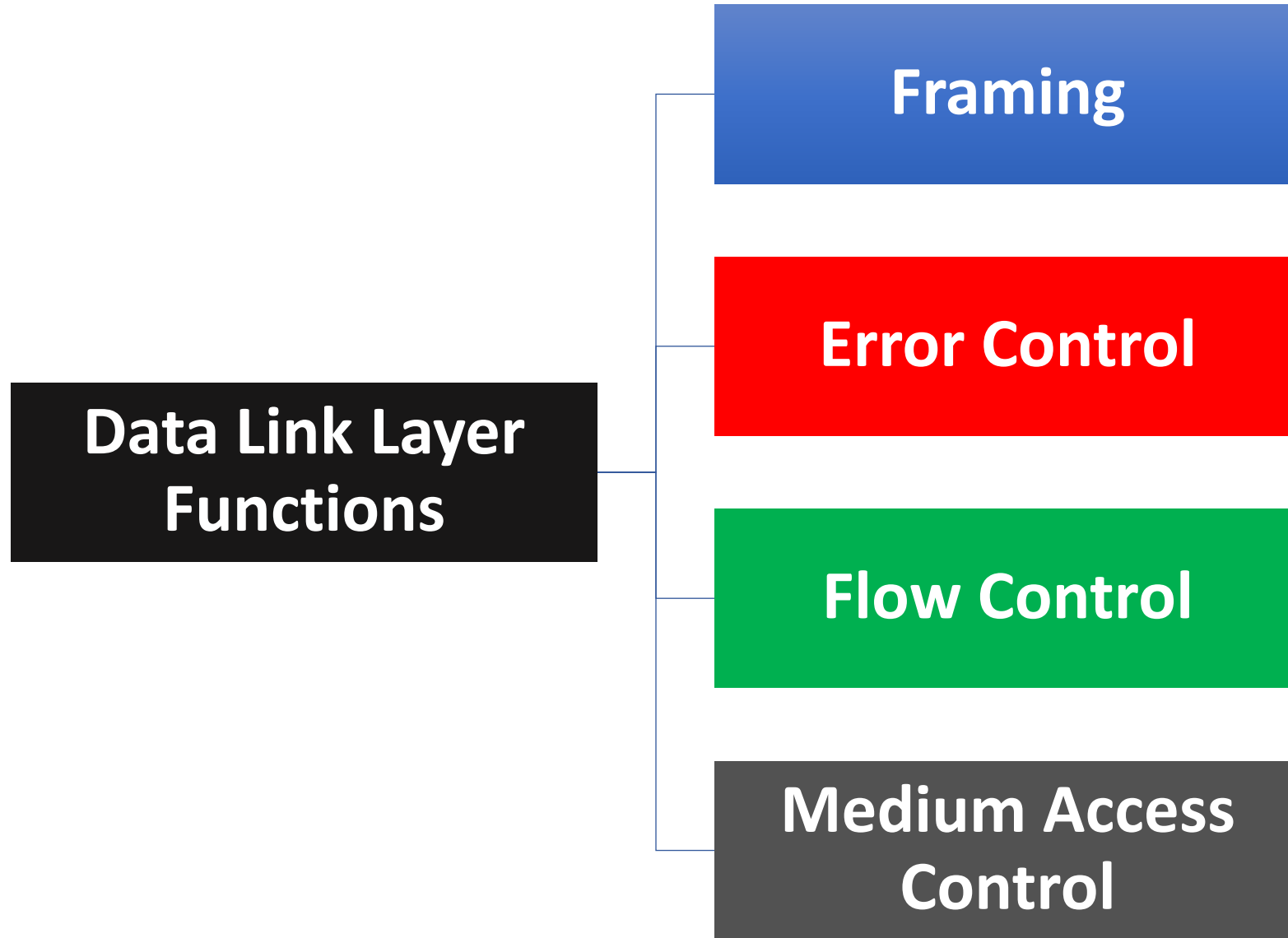


CHAPTER 6

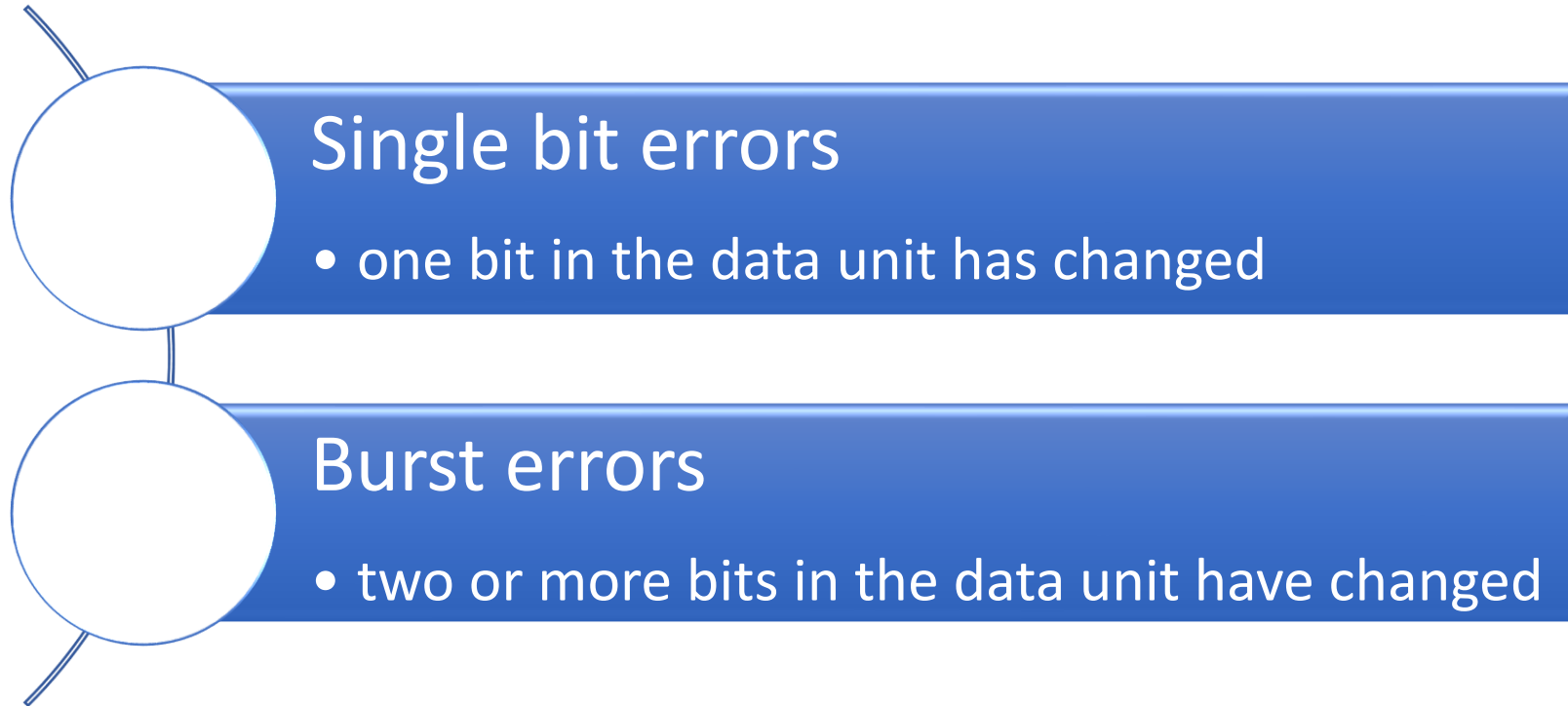
Error Detection and Correction



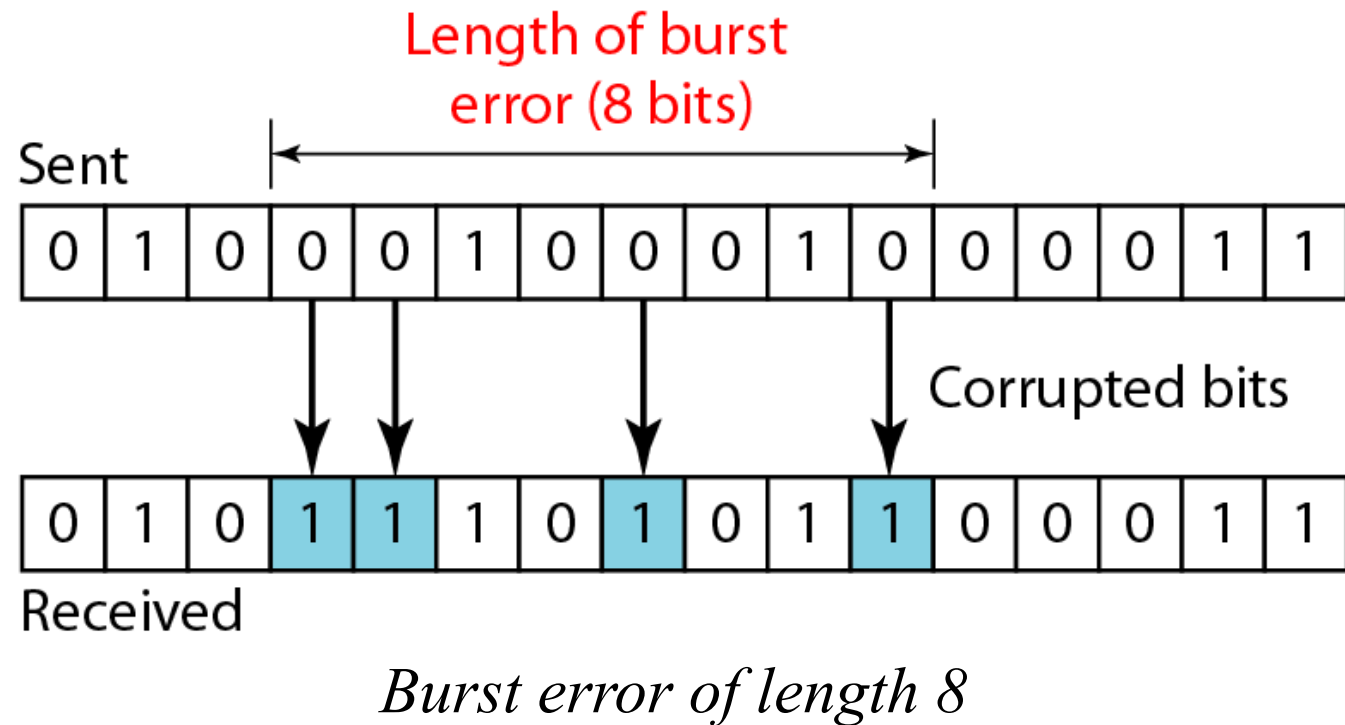
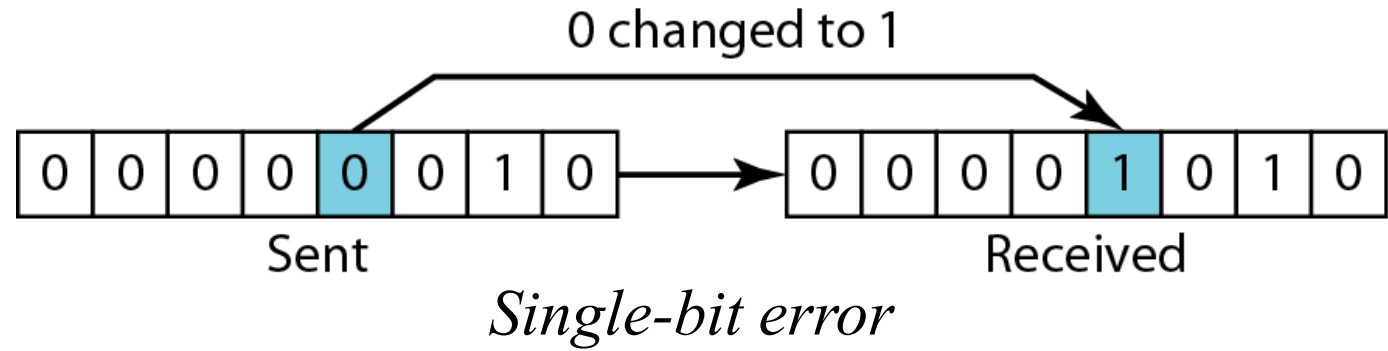
Error Control

Types of Errors

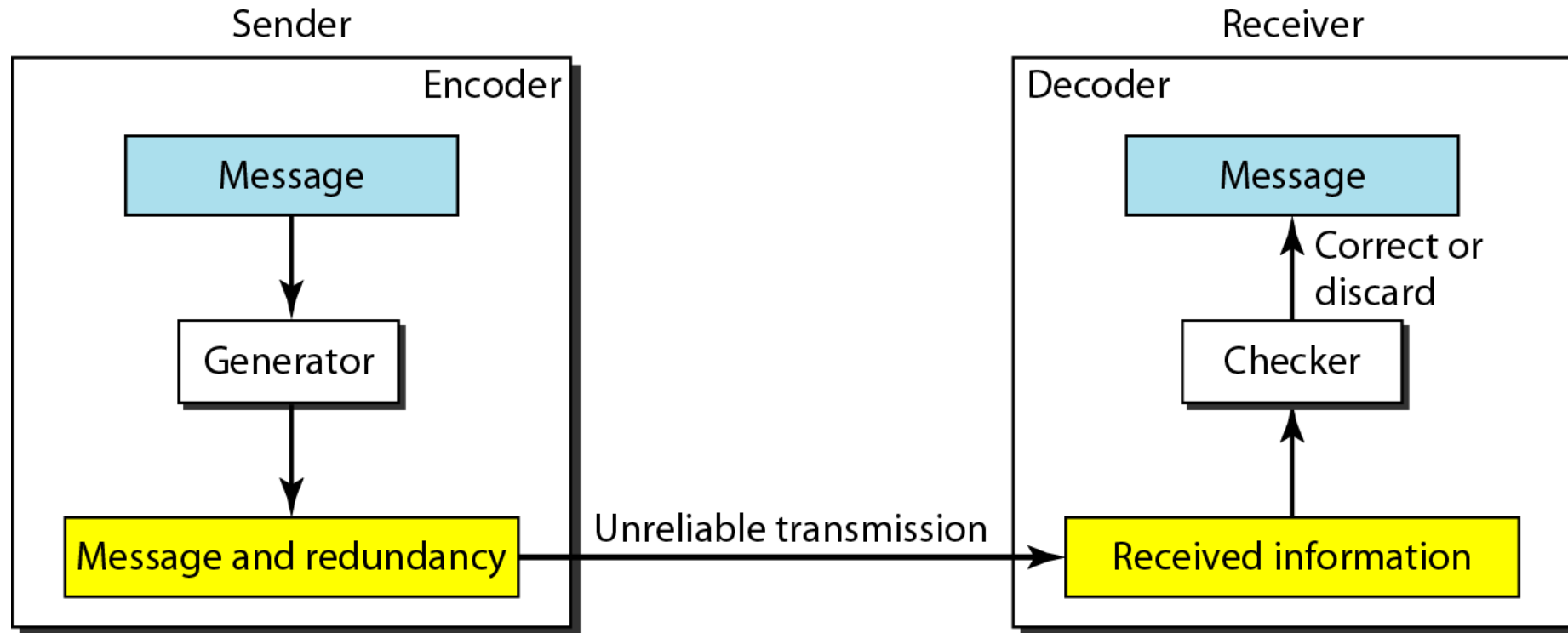
- Data can be corrupted during transmission.
- An error occurs when a bit is altered between transmission and reception



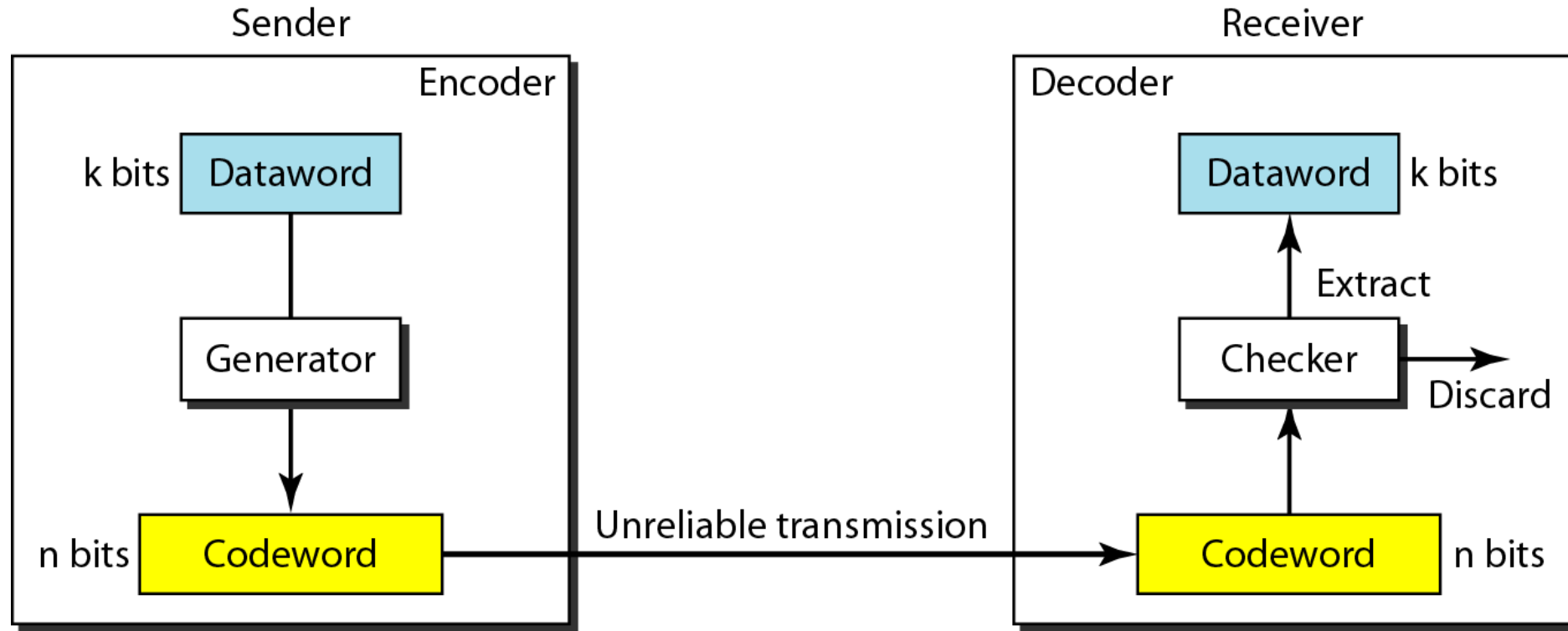
Types of Errors



The structure of encoder and decoder

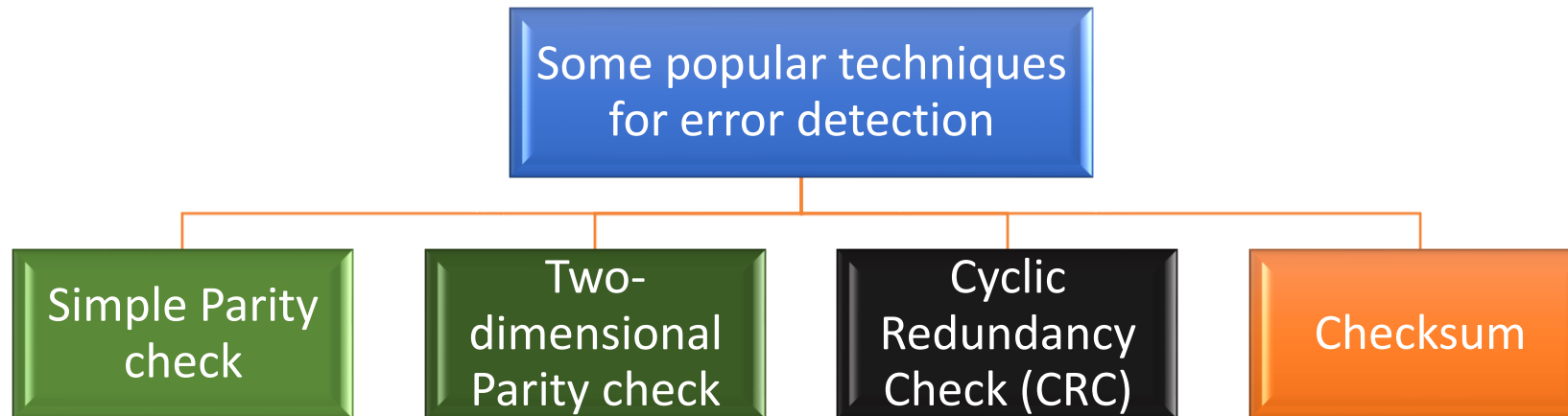


Process of error detection in block coding



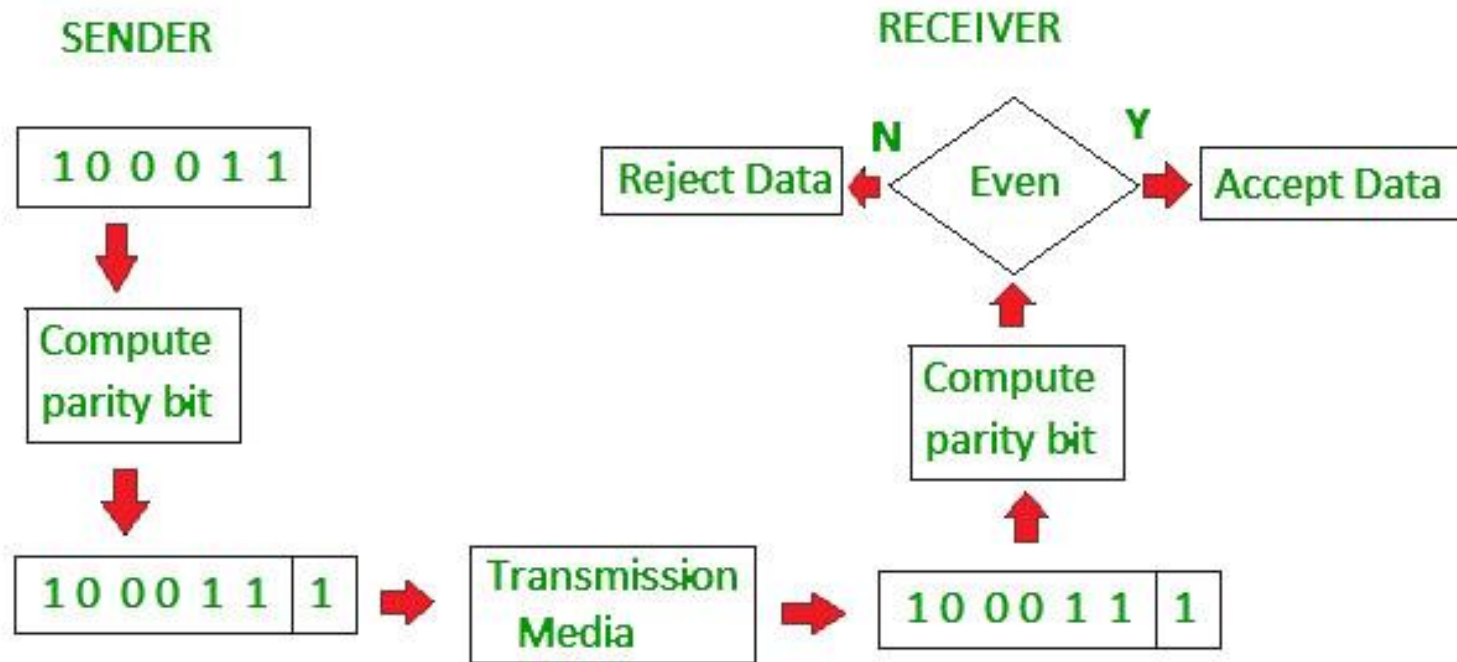
Error Detection

To detect or correct errors, we need to send extra (redundant) bits with data.

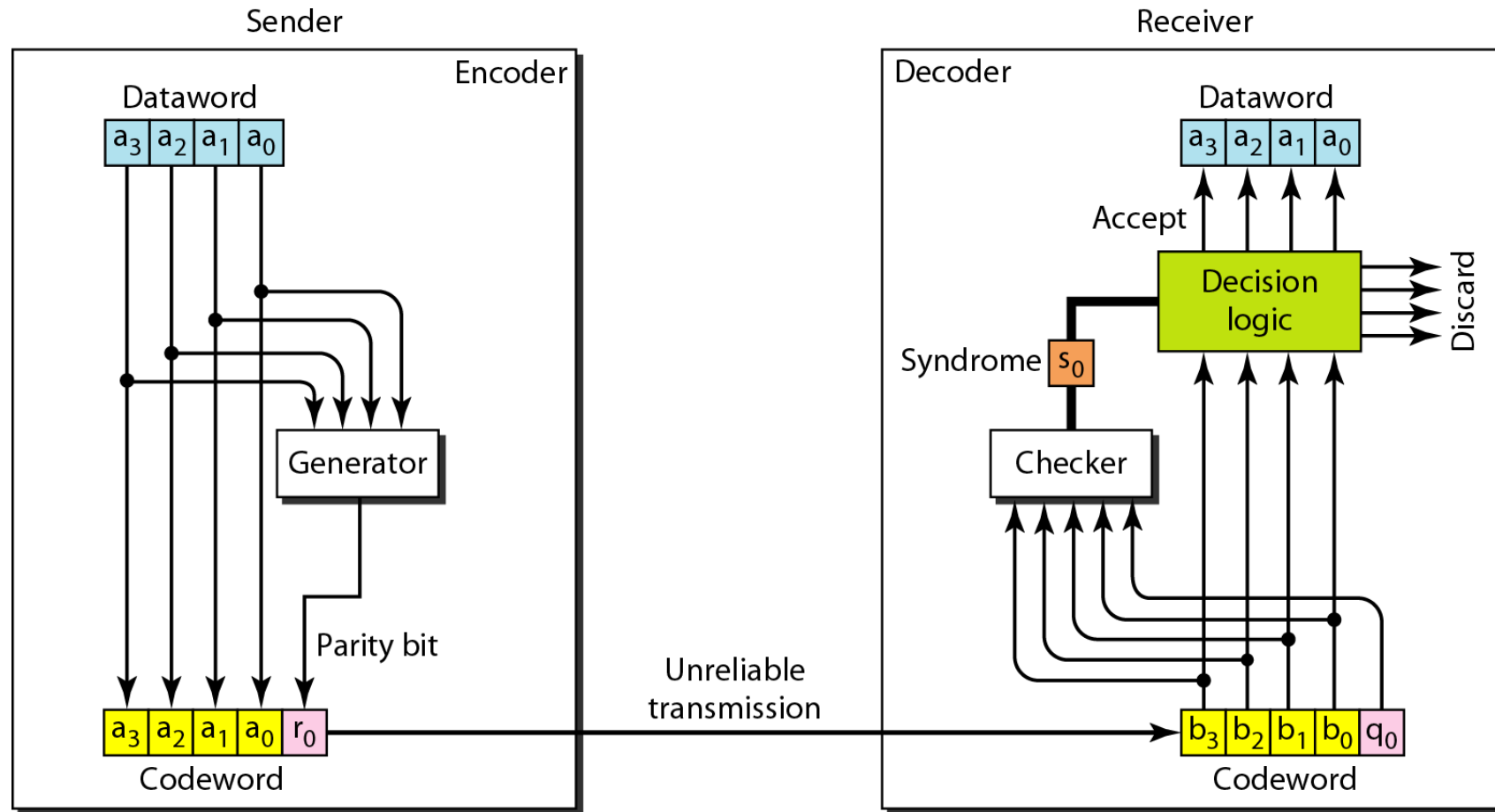


Simple Parity check

- 1 is added to the block if it contains odd number of 1's, and
- 0 is added if it contains even number of 1's



Encoder and decoder for simple parity-check code



Example

Suppose the sender wants to send the word “**World**”. In ASCII the five characters are coded as

W	o	r	l	d
1110111	1101111	1110010	1101100	1100100

The following shows the actual bits sent

1110111 <u>0</u>	1101111 <u>0</u>	1110010 <u>0</u>	1101100 <u>0</u>	1100100 <u>1</u>
------------------	------------------	------------------	------------------	------------------

Now suppose the word world is received by the receiver as:

1101111 <u>0</u>	1110010 <u>0</u>	1101100 <u>0</u>	1100100 <u>1</u>	1110111 <u>0</u>
------------------	------------------	------------------	------------------	------------------

The receiver counts the 1s in each character and comes up with even numbers: (6, 6, 4, 4, 4). **The data are accepted.**

Example

Now suppose the word world in previous Example is corrupted during transmission.

11111110 11011110 11101100 11011000 11001001

The receiver counts the 1s in each character and comes up with even and odd numbers: (7, 6, 5, 4, 4).

The receiver knows that the data are corrupted, discards them, and asks for retransmission.

Example

*Let us look at some transmission scenarios. Assume the sender sends the dataword **1011**. The codeword created from this dataword is **10111**, which is sent to the receiver. We examine five received codewords*

- 1. 10111*
- 2. 10**0**11*
- 3. 1011**0***
- 4. **00**11**0***
- 5. 01011*

Example

a_3	a_2	a_1	a_0	r_0
-------	-------	-------	-------	-------

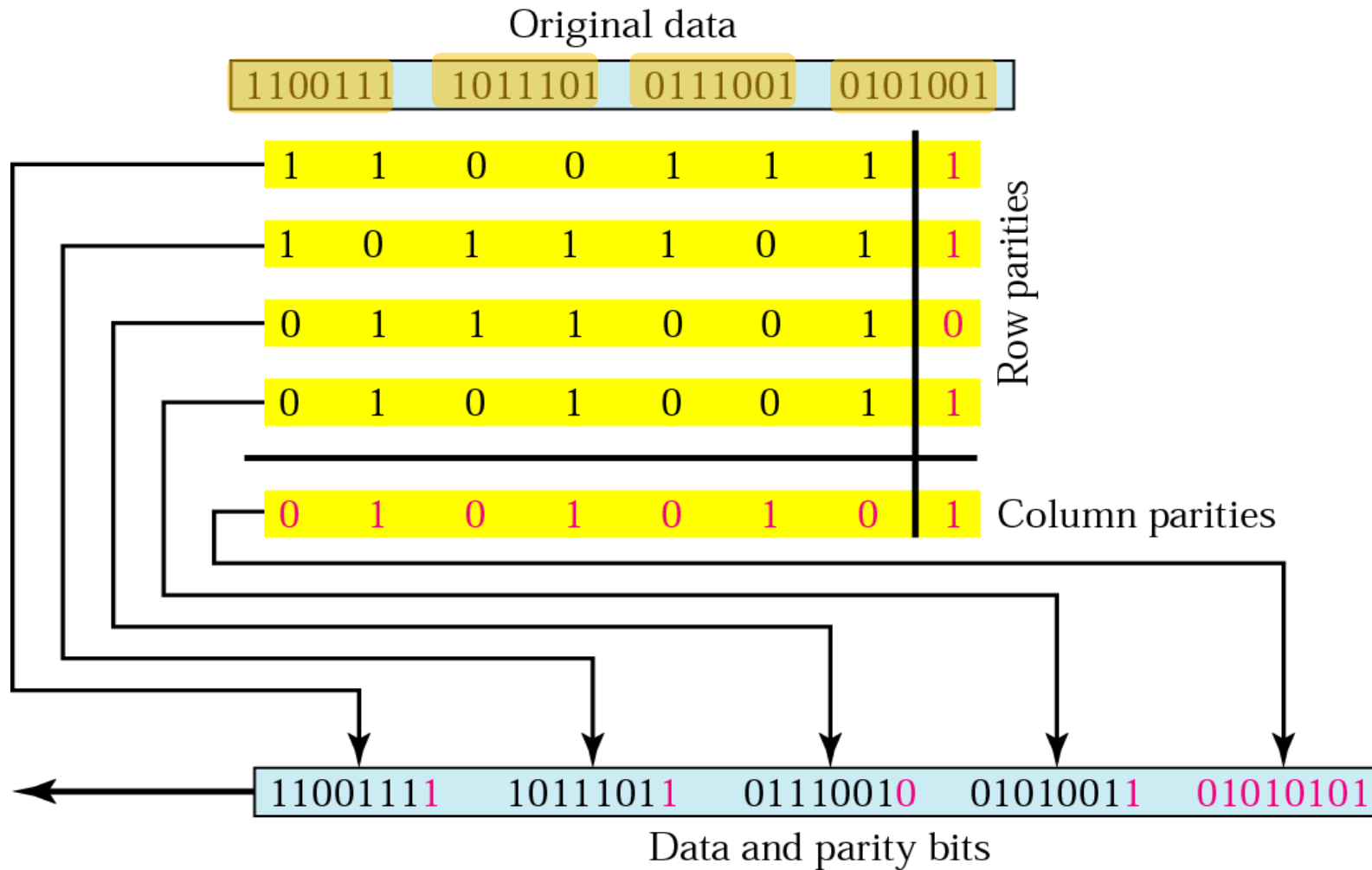
<i>dataword</i>	<i>codeword</i>	<i>error</i>	<i>syndrome</i>	<i>dataword</i>
<i>1011</i>	<i>10111</i>	<i>No error</i>	0	<i>1011</i>
<i>1011</i>	<i>10011</i>	a_1	1	Discard
<i>1011</i>	<i>10110</i>	r_0	1	Discard
<i>1011</i>	<i>00110</i>	$a_3 \quad r_0$	0 (# of ones even)	1011 (error)
<i>1011</i>	<i>01011</i>	$a_3, a_2, \text{ and } a_1$	1	Discard

A simple parity-check code can detect an odd number of errors.

Two-dimensional Parity check

- Parity check bits are calculated for each row, which is equivalent to a simple parity check bit.
- Parity check bits are also calculated for each columns, then both are sent along with the data.
- At the receiving end these are compared with the parity bits calculated on the received data.

Two-dimensional Parity check



Example

Sent

1	0	1	0	1	0	0	
0	0	1	1	1	0	0	
1	1	0	1	1	1	0	
1	1	1	0	0	1	1	

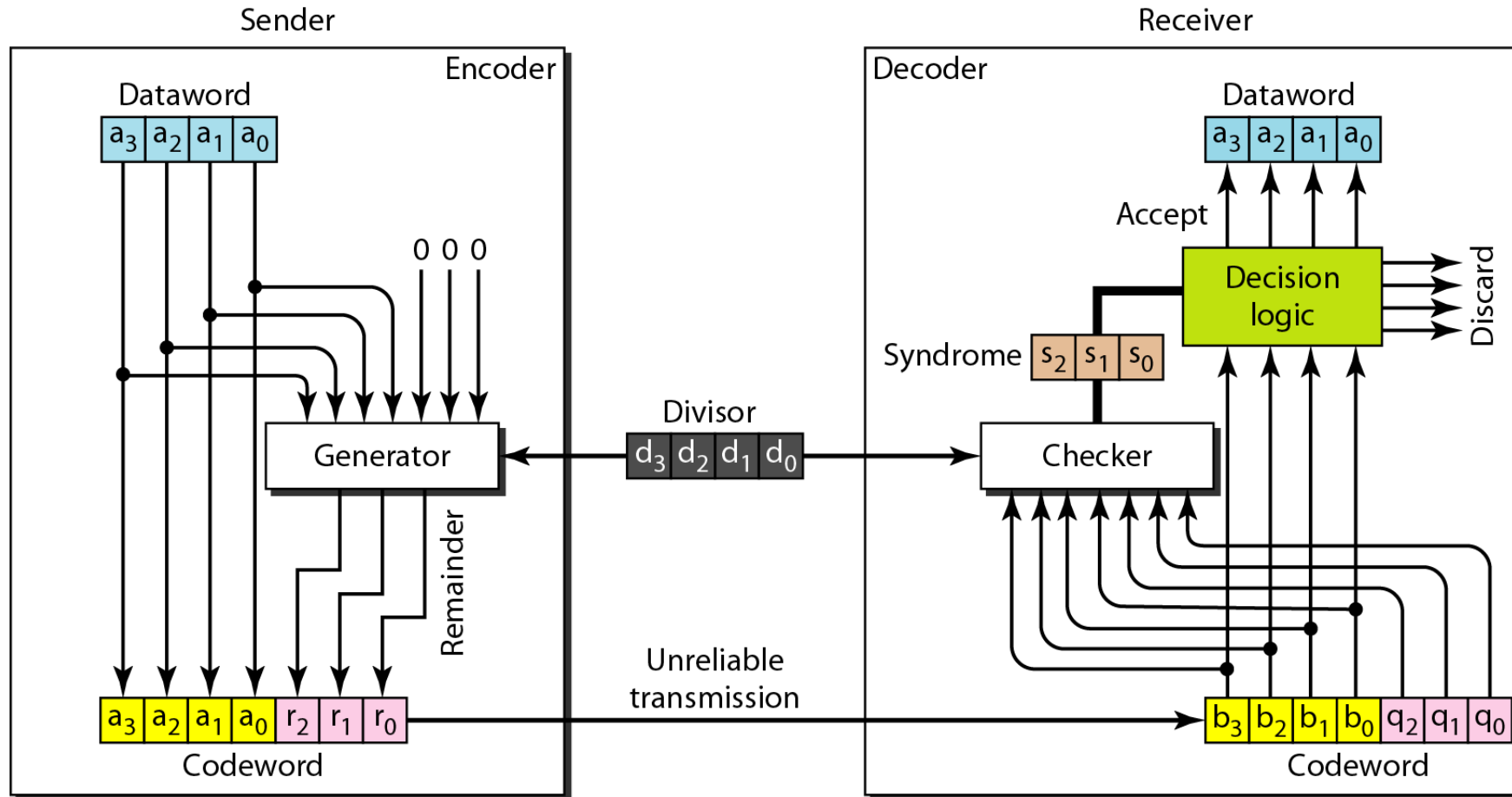
Received

1	0	1	0	1	0	0	1	
0	0	1	1	1	0	0	1	
1	1	0	1	1	1	0	1	
1	1	1	0	0	1	1	1	
1	0	1	0	1	0	1	0	

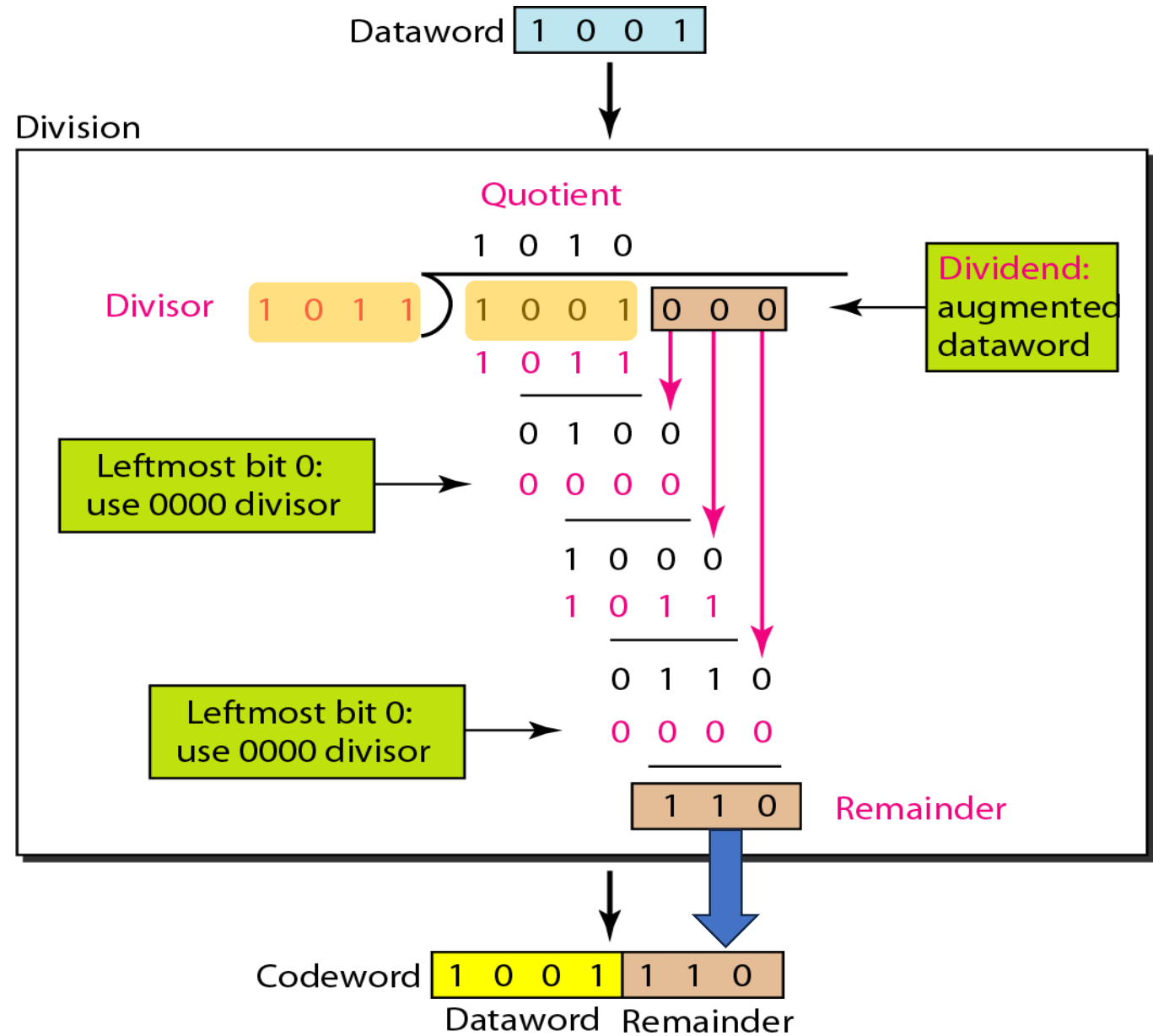
Cyclic Redundancy Check (CRC)

- One of the most common and powerful error-detecting codes
- A string of n 0s is added to the data unit.. The number n is one smaller than the number of bits in the fixed divisor
- The new data unit is divided by a divisor utilizing a procedure known as binary division; the remainder appearing from the division is CRC.
- Receiver divides the incoming frame by that number
 - If there is no remainder, assume there is no error

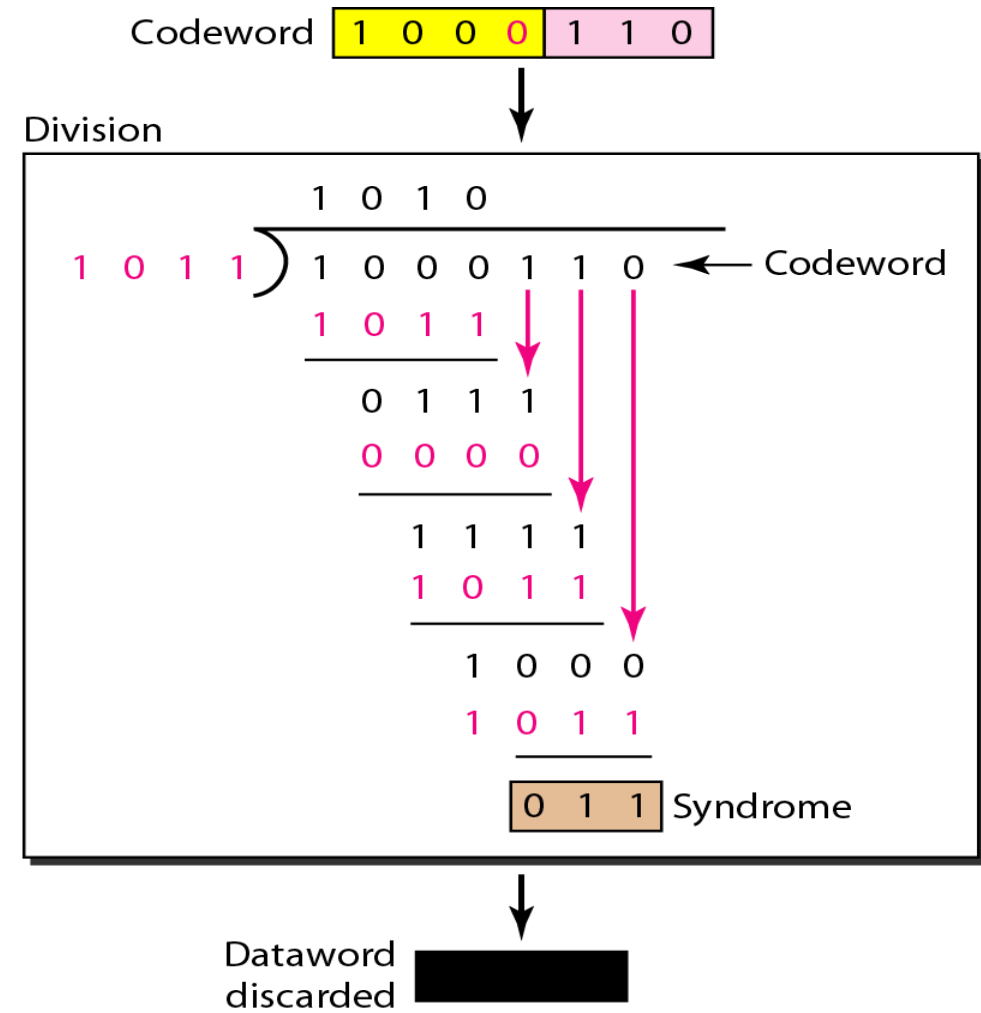
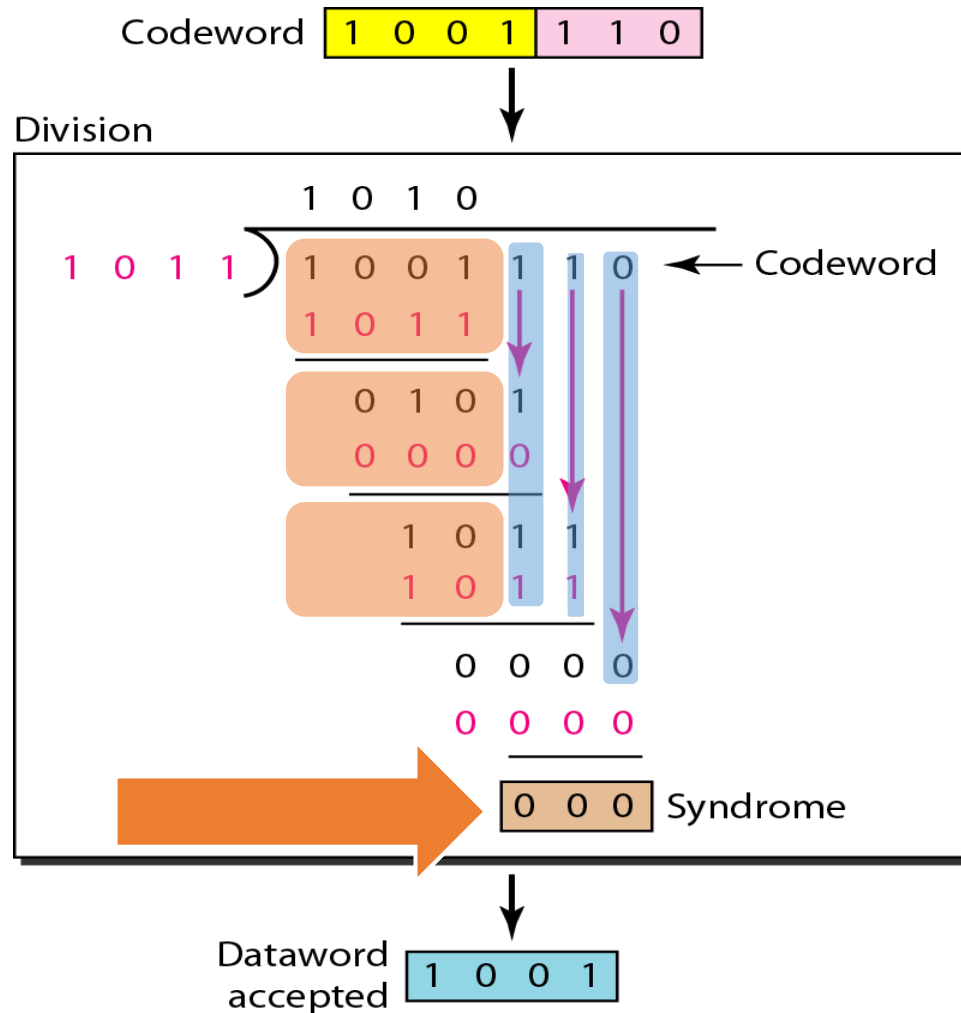
CRC Encoder and Decoder



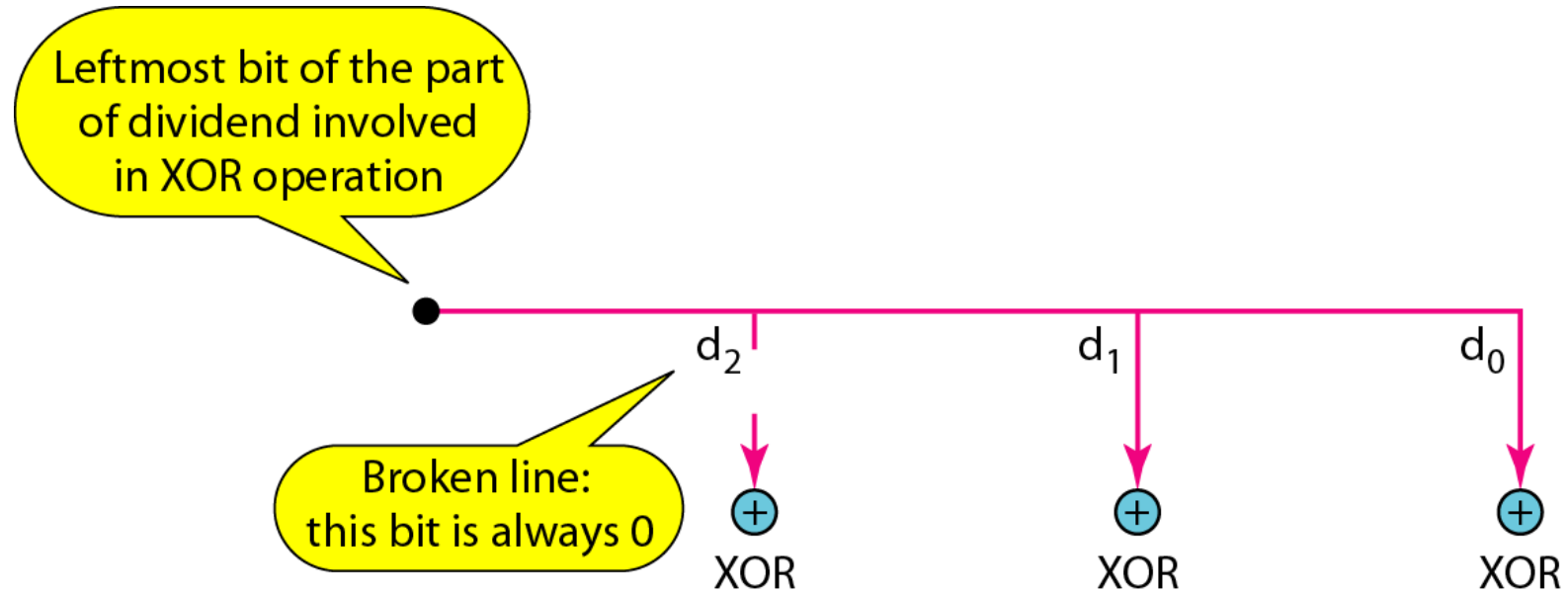
Division in CRC encoder



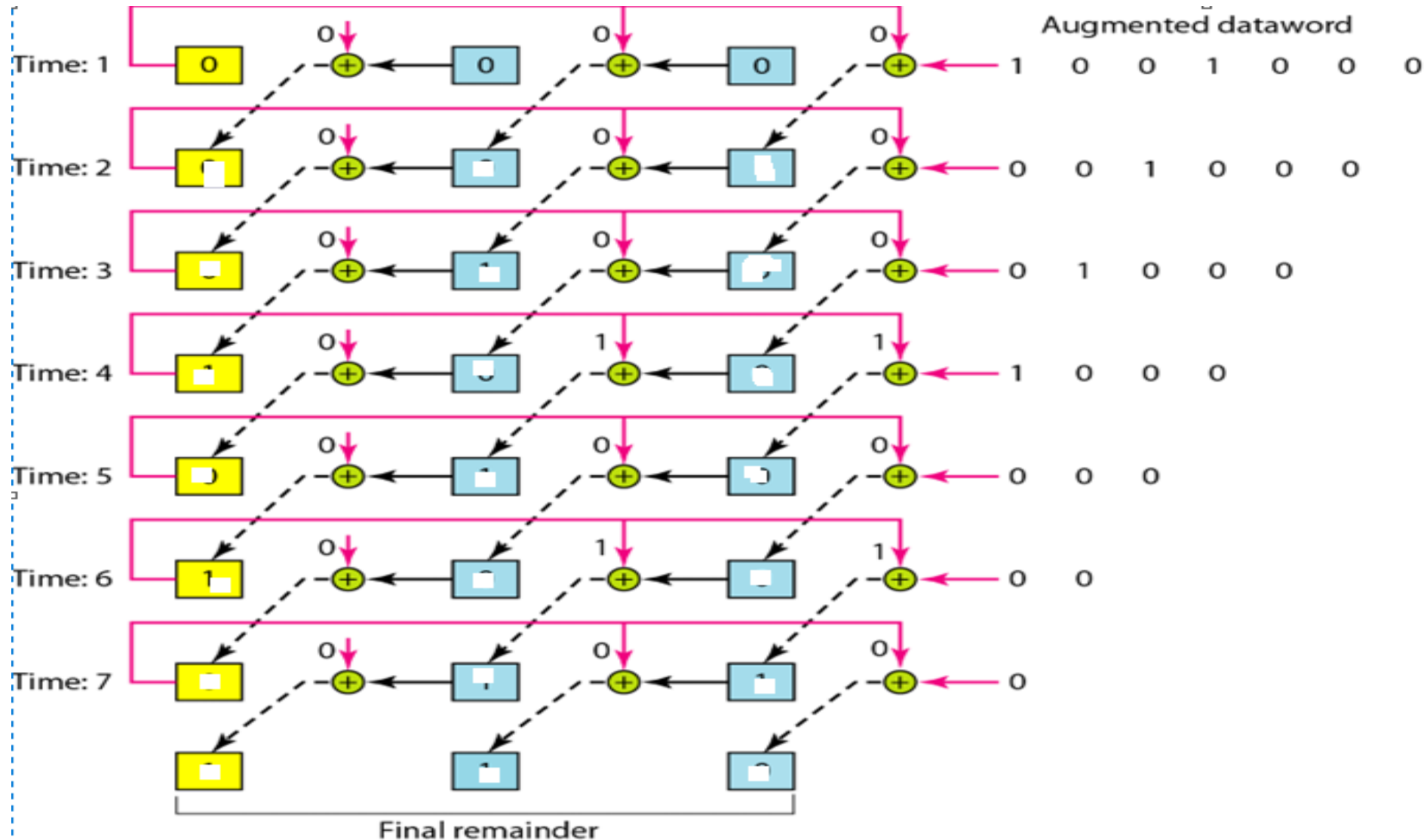
Division in the CRC decoder for two cases



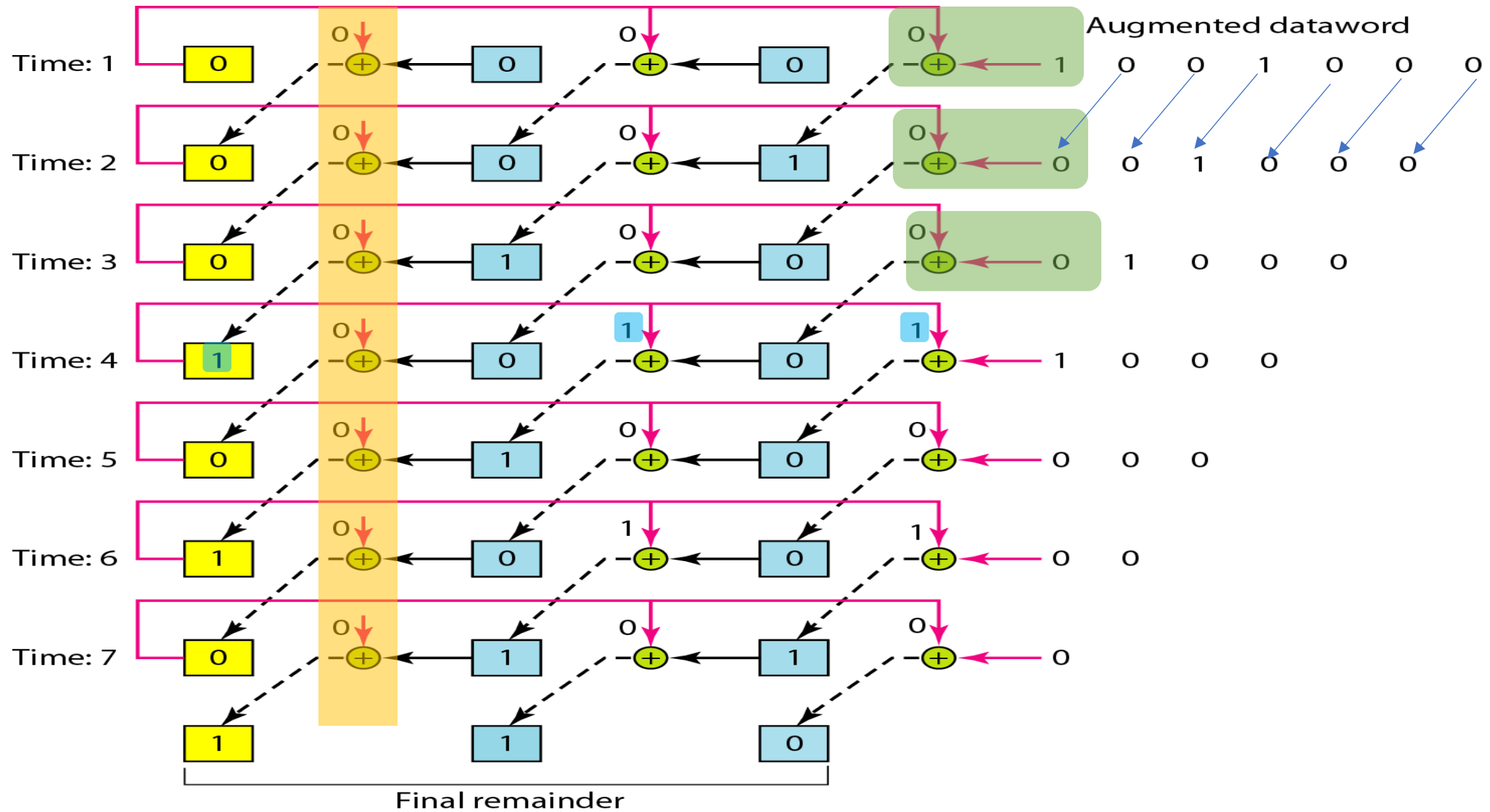
Hardwired design of the divisor in CRC



Simulation of division in CRC encoder

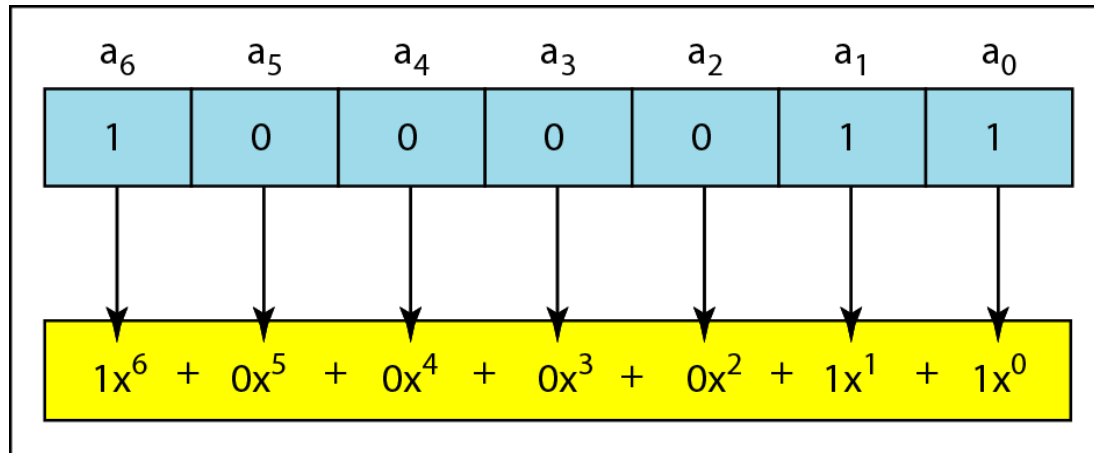


Simulation of division in CRC encoder

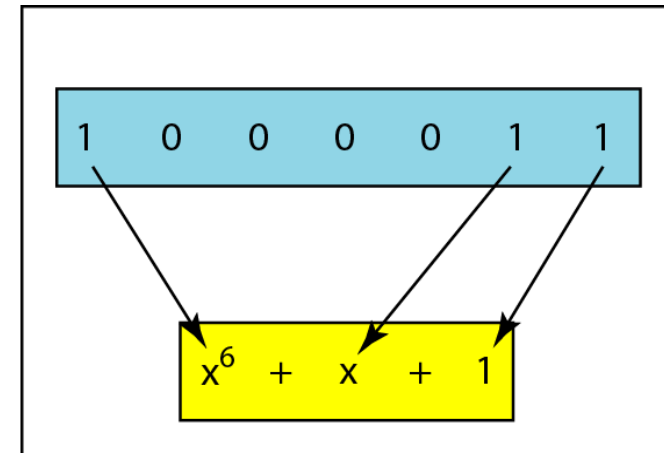


CRC Division using Polynomials

Polynomial to represent a binary word

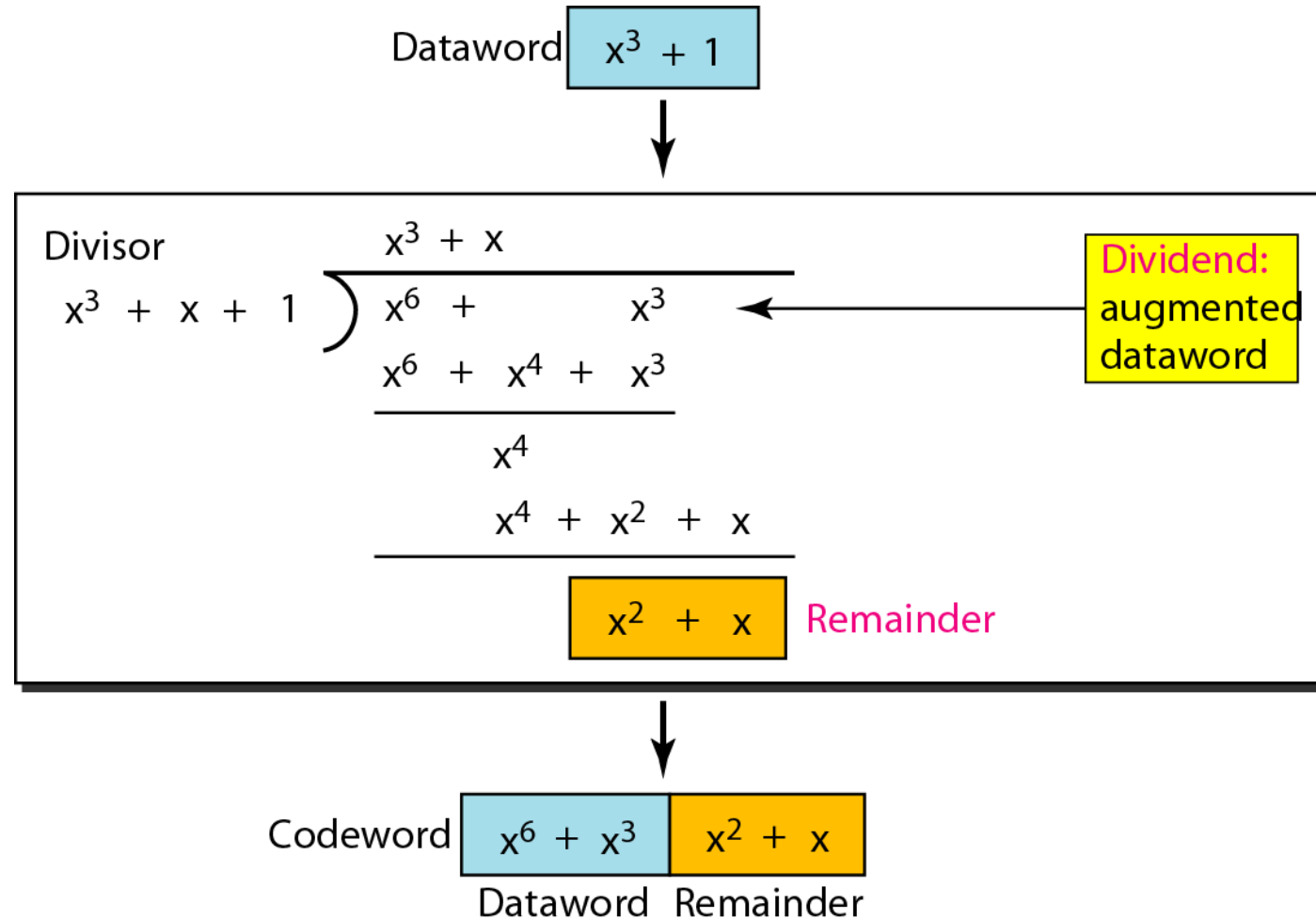


a. Binary pattern and polynomial



b. Short form

CRC Division using Polynomials



Note

The divisor in a cyclic code is normally called the generator polynomial or simply the generator.

If the generator has more than one term and the coefficient of x^0 is 1, all single errors can be caught.

If $s(x) \neq 0$, one or more bits is corrupted.

If $s(x) = 0$, either

- a.** No bit is corrupted. or
 - b.** Some bits are corrupted, but the decoder failed to detect it.
-

CRC Division using Polynomials

CRC Can detect all burst errors that affect an odd number of bits.

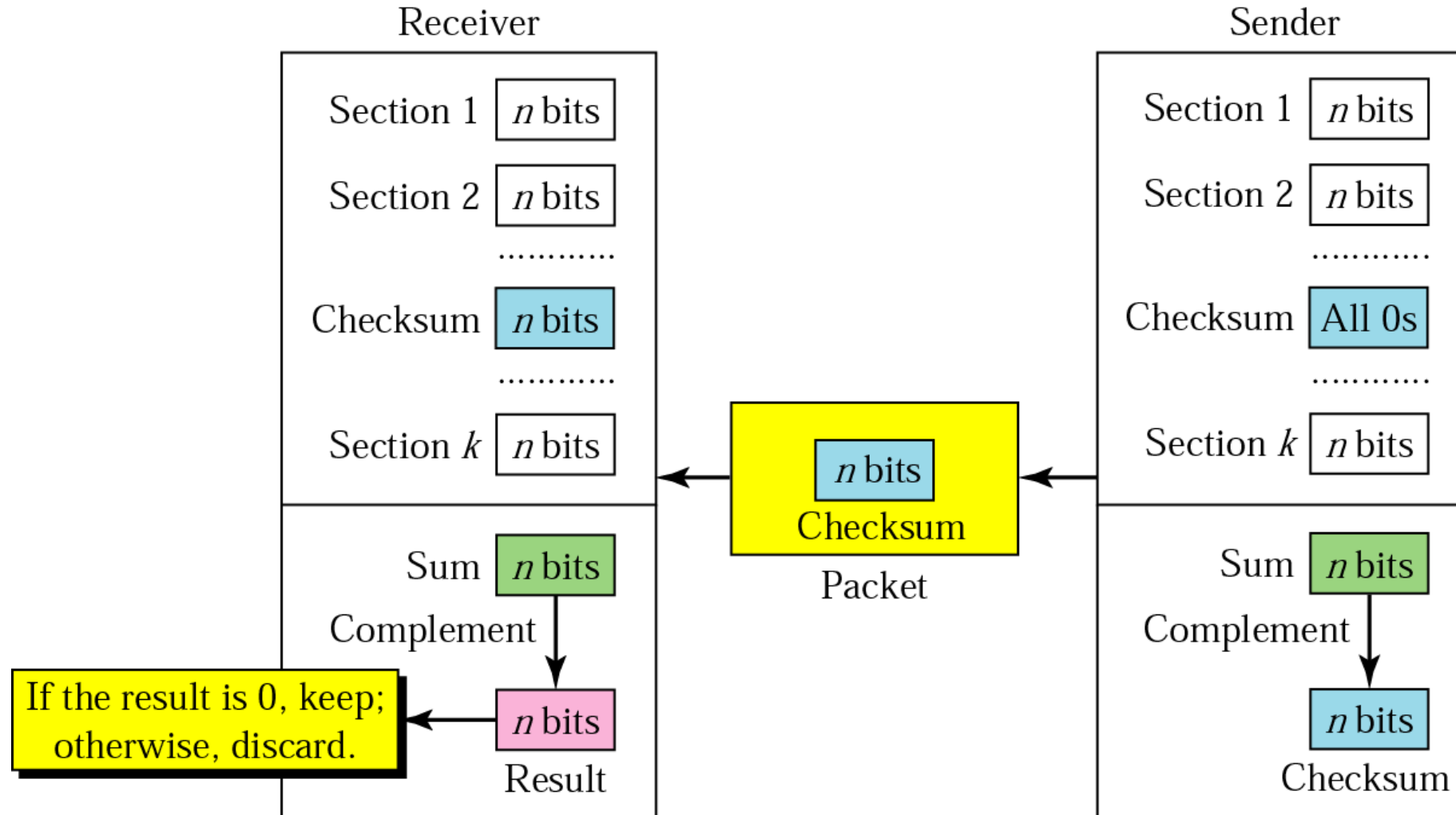
The CRC-12: $x^{12} + x^{11} + x^3 + x + 1$ which has a degree of 12, will detect:

- **all burst errors affecting an odd number of bits,**
- **will detect all burst errors with a length less than or equal to 12**

The Internet Checksum

- The checksum is used in the **network layer**.
- The data is divided into k segments each of m bits.
- In the sender's end the segments are added. The sum is complemented (1's complement) to get the checksum.
- The checksum segment is sent along with the data segments.
- At the receiver's end, all received segments are added. The sum is complemented.
- If the result is zero, the received data is accepted; otherwise discarded.

CHECKSUM



Example

Suppose the following block of 16 bits is to be sent using a checksum of 8 bits.

10101001 00111001

The numbers are added

1 0 1 0 1 0 0 1

0 0 1 1 1 0 0 1

Sum

1 1 1 0 0 0 1 0

Using one's complement, Checksum 00011101

The pattern sent is 10101001 00111001 00011101

Example

*Now suppose the receiver receives the pattern sent and there is no error.
10101001 00111001 00011101*

When the receiver adds the three sections, it will get all 1s, which, after complementing, is all 0s and shows that there is no error.

	1	0	1	0	1	0	0	1
	0	0	1	1	1	0	0	1
	0	0	0	1	1	1	0	1
Sum	1	1	1	1	1	1	1	1

Complement 00000000 means *that the pattern is OK.*

Example

Now suppose there is a burst error that affects 4 bits.

10101111 11111001 00011101

When the receiver adds the three sections, it gets

	1 0 1 0 1 1 1 1
	1 1 1 1 1 0 0 1
	0 0 0 1 1 1 0 1
Partial Sum	1 1 1 0 0 0 1 0 1
Carry	1
Sum	1 1 0 0 0 1 1 0

Complement 00111001 the pattern is corrupted.

Example

Original Data

10011001	11100010	00100100	10000100
----------	----------	----------	----------

1
k=4, m=8

2

3

4

Sender

1 10011001
2 11100010

① 01111011
1

3 01111100
00100100

4 10100000
10000100

① 00100100
1

Sum: 00100101

Checksum: 11011010

Receiver

1 10011001
2 11100010

① 01111011
1

3 01111100
00100100

4 10100000
10000100

① 00100100
1

00100101
11011010

Sum: 11111111

Complement: 00000000

Conclusion: Accept Data

Example

- Consider a header that consists of 10 octets, with the checksum in the last two octets with the following content (in hexadecimal):

00 01 F2 03 F4 F5 F6 F7 00 00

- The transmitted packet is

00 01 F2 03 F4 F5 F6 F7 **22 0D**

Sender

	0	0	0	1
	F	2	0	3
	F	4	F	5
	F	8	F	7
2	D	D	F	0
Carry				2
sum	D	D	F	2
	2	2	F	D
checksum				

Receiver

	0	0	0	1
	F	2	0	3
	F	4	F	5
	F	8	F	7
2	D	D	F	0
Carry				2
sum	D	D	F	2
checksum	2	2	F	D
	F	F	F	F
	0	0	0	0

Hamming Code

- To calculate Hamming Error Correction Code (ECC):
 - Number bits from 1 on the left
 - All bit positions that are a power 2 are parity bits
 - Each parity bit checks certain data bits:
 - P1 checks bits (1,3,5,7,9,11,...)
 - P2 checks bits (2,3,6,7,10,11,14,15,...)
 - P4 checks bits (4–7, 12–15, 20–23,...)
 - P8 checks bits (8–15, 24–31, 40–47,...)

Bit position		1	2	3	4	5	6	7	8	9	10	11	12
Encoded data bits		p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8
Parity bit coverate	p1	X		X		X		X		X		X	
	p2		X	X			X	X			X	X	
	p4				X	X	X	X					X
	p8								X	X	X	X	X

Hamming Code

Consider the 8-bit data word 11000100. We include four parity bits with it and arrange the 12 bits as follows:

Bit position:

1	2	3	4	5	6	7	8	9	10	11	12
P ₁	P ₂	1	P ₄	1	0	0	P ₈	0	1	0	0

$$P_1 = \text{XOR of bits}(3,5,7,9,11) = 1 \oplus 1 \oplus 0 \oplus 0 \oplus 0 = 0$$

$$P_2 = \text{XOR of bits}(3,6,7,10,11) = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 = 0$$

$$P_4 = \text{XOR of bits}(5,6,7,12) = 1 \oplus 0 \oplus 0 \oplus 0 = 1$$

$$P_8 = \text{XOR of bits}(9,10,11,12) = 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

Hamming Code

- The data is sent to the receiver together with the parity bit as 12-bit composite word.

1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	1	0	0	1	0	1	0	0

- In the receiver, the parity is checked over the same combination of bits including the parity bit.

$$C_1 = \text{XOR of bits}(1,3,5,7,9,11) = 0 \rightarrow \text{two ones}$$

$$C_2 = \text{XOR of bits}(2,3,6,7,10,11) = 0 \rightarrow \text{two ones}$$

$$C_4 = \text{XOR of bits}(4,5,6,7,12) = 0 \rightarrow \text{two ones}$$

$$C_8 = \text{XOR of bits}(8,9,10,11,12) = 0 \rightarrow \text{two ones}$$

Error-Detection

- Since the bits were stored with even parity, the result,

$C = C_8C_4C_2C_1 = 0000$, indicates that no error has occurred.

- If $C \neq 0$, then the 4-bit binary number formed by the check bits gives the position of the erroneous bit.

Example

Bit position: 1 2 3 4 5 6 7 8 9 10 11 12

0 0 1 1 1 0 0 1 0 1 0 0 No error

1 0 1 1 1 0 0 1 0 1 0 0 Error in bit 1

0 0 1 1 **0** 0 0 1 0 1 0 0 Error in bit 5

- Evaluating the XOR of the corresponding bits, get the four check bits

	C_8	C_4	C_2	C_1
For no error:	0	0	0	0
with error in bit 1:	0	0	0	1
with error in bit 5:	0	1	0	1

Exercise

12-bit Hamming codeword (000011101010) containing data and parity bits, what was the original corrected data word?